log_data_analysis

December 13, 2025

1 Log Data Analysis Codebook

This notebook performs all of the analysis reported in the results section of the paper. It is structured into the following main sections: 1. Setup and loading data 2. Calculating reliability of the codebook 3. Generating the codebook 4. Summary statistics 5. Statistics for the categorisation

1.1 1. Setup and loading data

We begin by importing necessary modules and loading the data. This uses a Docker container that contains the log data inside of it. This is not directly replicable as we did not have permission to publish students' log data.

[3]:
```
from analysis.loading_services.get_data import *
from analysis.core_filter_functions import *
from analysis.filter_functions import *
from analysis.stats_functions import *
from analysis.stats_helper_functions import *
from analysis.codebook_analysis import *

from latextable import draw_latex
import texttable
#from summary_stats_functions import *

coded_program_logs: DataFrame = get_data("coded_program_logs")
codes: list[str] = get_data("codes")["code_name"].values.tolist()
participants: DataFrame = filter_null_data(get_data("participants"),␣
 ↪"participants")
program_logs: DataFrame =␣
 ↪clear_unattempted_exercises(filter_null_data(get_data("program_logs"),"program_logs"))
participants_list: list[str] = participants['participantid'].values.tolist()

dataframes = {
    "coded_program_logs": coded_program_logs,
    "codes": codes,
    "participants": participants,
    "program_logs": program_logs
}
```

1.2 2. Calculating reliability of the codebook

See `categorisation_irr_comparison.csv` for the coding that was performed for IRR values.

1.3 3. Generating the codebook

Now the codebook that is reported in Appendix of the paper is generated. See `results/codebook_frequency.csv` for the resultant codebook and frequencies.

[4]:
```
save_code_counts()
print("Codebook saved to results/codebook_frequency.csv")

print(f"Number of codes: ␣
 ↪{len(flatten_codebook_tree(create_codebook_with_frequencies()))}\n")

most_frequent_bottom_level_codes_per_exercise_attempt: list[tuple[str, int]] = ␣
 ↪get_most_frequent_bottom_level_codes_per_exercise_attempt()
print(f"Most frequently bottom level codes per exercise attempt: ␣
 ↪(n={N_ATTEMPTS})")
for label, count in most_frequent_bottom_level_codes_per_exercise_attempt:
    print(f"- {label}: {count} ({count / 322 * 100:.2f}%)")

most_frequent_bottom_level_codes_per_participant: list[tuple[str, int]] = ␣
 ↪get_most_frequent_bottom_level_codes_per_participant()
print(f"\nMost frequently bottom level codes per participant: ␣
 ↪(n={N_PARTICIPANTS})")
for label, count in most_frequent_bottom_level_codes_per_participant:
    print(f"- {label}: {count} ({count / 73 * 100:.2f}%)")
```

```
Codebook saved to results/codebook_frequency.csv
Number of codes: 77

Most frequently bottom level codes per exercise attempt: (n=322)
- First change on line referred to in error message: 257 (79.81%)
- Ran code before making changes: 212 (65.84%)
- Reverted previous changes: 159 (49.38%)
- Correctly resolved syntax error: 148 (45.96%)
- Inconsequential changes to whitespace of program: 136 (42.24%)
- Repeated unsuccessful runs with no changes: 135 (41.93%)
- Made changes before running code: 110 (34.16%)
- Correctly resolved logical error while code not running: 103 (31.99%)
- Inconsequential changes to (existing) other statements: 91 (28.26%)
- Repeated successful runs when code successfully runs: 89 (27.64%)
- Inconsequential changes to (existing) outputs: 69 (21.43%)
- Incorrect syntactical changes to output statements: 67 (20.81%)
- Correctly resolved runtime error: 67 (20.81%)
- Incorrect syntactical changes to variable assignments: 66 (20.50%)
- Incorrect syntactical changes to other symbols: 64 (19.88%)
```

```
Most frequently bottom level codes per participant: (n=73)
- First change on line referred to in error message: 71 (97.26%)
- Ran code before making changes: 69 (94.52%)
- Inconsequential changes to whitespace of program: 64 (87.67%)
- Reverted previous changes: 61 (83.56%)
- Correctly resolved syntax error: 59 (80.82%)
- Inconsequential changes to (existing) other statements: 55 (75.34%)
- Repeated unsuccessful runs with no changes: 55 (75.34%)
- Correctly resolved logical error while code not running: 53 (72.60%)
- Repeated successful runs when code successfully runs: 49 (67.12%)
- Inconsequential changes to (existing) outputs: 48 (65.75%)
- Made changes before running code: 47 (64.38%)
- Correctly resolved runtime error: 43 (58.90%)
- Incorrect syntactical changes to output statements: 42 (57.53%)
- Incorrect syntactical changes to variable assignments: 41 (56.16%)
- Incorrect syntactical changes involving selection: 39 (53.42%)
```

Generating the table in LaTex

[ ]:
```
most_frequent_codes_table = texttable.Texttable()
most_frequent_codes_table.set_cols_align(["l", "c", "c", "l", "c", "c"])

most_frequent_codes_table.add_row(["Behaviour", "$n_{students}$", "\%",␣
 ↪"Behaviour", "$n_{attempts}$", "\%"])
for (label_attempt, count_attempt), (label_participant, count_participant) in␣
 ↪zip(most_frequent_bottom_level_codes_per_exercise_attempt,␣
 ↪most_frequent_bottom_level_codes_per_participant):
    pct_attempts = count_attempt / N_ATTEMPTS * 100
    pct_participants = count_participant / N_PARTICIPANTS * 100
    most_frequent_codes_table.add_row([label_participant, count_participant,␣
 ↪f"{pct_participants:.0f}", label_attempt, count_attempt, f"{pct_attempts:.
 ↪0f}"])

print(draw_latex(most_frequent_codes_table,
                 caption="The 15 most common behaviours observed across␣
 ↪students and exercise attempts",
                 label="tab:most-common-behaviours",
                 position="h",
                 use_booktabs=True))
```

1.4 4. Summary statistics

Some of these were used to generate some of the tables in the paper.

[6]:
```
print(f"Number of attempted exercises:␣
 ↪{get_number_attempted_exercises(filtered_ids={}, **dataframes)}")
print(f"Total number of runs: {total_number_of_runs(filtered_ids={},␣
 ↪**dataframes)} \n")
```

```
print("Median number of runs for each attempted exercise:")
for i in range(0, 5):
    print(f"- Exercise {i+1}:␣
 ↪{quartiles_for_exercise(get_number_runs(filtered_ids={}, **dataframes))[i]}␣
 ↪(skewness = {skewness_for_exercises(get_number_runs(filtered_ids={},␣
 ↪exercise_number=i, **dataframes))[i]})")
print(f"Overall: {quartiles_for_exercise(get_number_runs(filtered_ids={},␣
 ↪**dataframes))[5]} (skewness =␣
 ↪{skewness_for_exercises(get_number_runs(filtered_ids={}, exercise_number=i,␣
 ↪**dataframes))[5]})")

print("\nMedian amount of time spent on each attempted exercise (in seconds):")
for i in range(0, 5):
    print(f"- Exercise {i+1}:␣
 ↪{quartiles_for_exercise(get_time_on_exercises(filtered_ids={},␣
 ↪exercise_number=i, **dataframes))[i]} (skewness =␣
 ↪{skewness_for_exercises(get_time_on_exercises(filtered_ids={},␣
 ↪exercise_number=i, **dataframes))[i]})")
print(f"Overall: {quartiles_for_exercise(get_time_on_exercises(filtered_ids={},␣
 ↪**dataframes))[5]} (skewness =␣
 ↪{skewness_for_exercises(get_time_on_exercises(filtered_ids={},␣
 ↪**dataframes))[5]})")

print(f"\nNumber of students who ended every exercise in an error state:␣
 ↪{length_of_participants_list(filtered_ids=filter_by_students_who_ended_all_attempted_exerci
 ↪**dataframes)}")
print(f"Number of students who ended the exercises in a correct state:
 ↪{get_end_state_of_program(filtered_ids={}, **dataframes)}\n")
print(f"Number of students who ended every exercise in a correct state:␣
 ↪{length_of_participants_list(filtered_ids=filter_by_students_who_ended_all_attempted_exerci
 ↪**dataframes)}")
print(f"Number of exercise attempts where every run was unsuccessful:␣
 ↪{get_number_exercises_in_error_state_for_every_run(filtered_ids={},␣
 ↪**dataframes)}\n")

print(f"Median edit distances between runs:␣
 ↪{quartiles_for_exercise(get_edit_distance_between_changed_runs(filtered_ids={},␣
 ↪**dataframes))} (skewness =␣
 ↪{skewness_for_exercises(get_edit_distance_between_changed_runs(filtered_ids={},␣
 ↪**dataframes))})")
print(f"Edit distance between first and last snapshot for each exercise:␣
 ↪{quartiles_for_exercise(get_edit_distance_between_first_and_last_snapshot(filtered_ids={},␣
 ↪**dataframes))} (skewness =␣
 ↪{skewness_for_exercises(get_edit_distance_between_first_and_last_snapshot(filtered_ids={},␣
 ↪**dataframes))})")
```

```
print(f"Median edit distance between first and last snapshot of each attempted␣
 ↪exercise:␣
 ↪{quartiles_for_exercise(get_edit_distance_between_first_and_last_snapshot(filtered_ids={},␣
 ↪**dataframes))} (skewness =␣
 ↪{skewness_for_exercises(get_edit_distance_between_first_and_last_snapshot(filtered_ids={},␣
 ↪**dataframes))})\n")
```

```
Number of attempted exercises: [72, 71, 68, 60, 51, 322]
Total number of runs: [938, 1335, 1254, 1055, 2628, 7210]

Median number of runs for each attempted exercise:
- Exercise 1: [1.0, 3.0, 7.0, 15.0, 85.0] (skewness = 2.54)
- Exercise 2: [1.0, 6.5, 13.0, 27.5, 72.0] (skewness = 1.38)
- Exercise 3: [2.0, 7.0, 14.0, 28.0, 81.0] (skewness = 1.48)
- Exercise 4: [1.0, 6.0, 13.0, 21.5, 65.0] (skewness = 1.35)
- Exercise 5: [2.0, 15.0, 35.0, 78.0, 218.0] (skewness = 1.56)
Overall: [1.0, 6.0, 13.0, 28.75, 218.0] (skewness = 3.33)

Median amount of time spent on each attempted exercise (in seconds):
- Exercise 1: [28.96, 91.5, 153.27, 287.08, 1183.83] (skewness = 2.33)
- Exercise 2: [28.72, 175.82, 304.44, 515.4, 1416.51] (skewness = 1.3)
- Exercise 3: [23.4, 225.0, 342.87, 560.35, 1569.28] (skewness = 1.54)
- Exercise 4: [56.74, 144.28, 250.67, 322.33, 737.55] (skewness = 1.15)
- Exercise 5: [51.72, 193.87, 290.14, 414.5, 1010.64] (skewness = 1.1)
Overall: [23.4, 142.91, 274.17, 442.76, 1569.28] (skewness = 1.61)

Number of students who ended every exercise in an error state: 26
Number of students who ended the exercises in a correct state:{'Ended exercise
with syntax error(s)': [7, 27, 21, 12, 8], 'Ended exercise with runtime
error(s)': [14, 5, 0, 16, 1], 'Ended exercise with type error(s)': [2, 4, 2, 23,
0], 'Ended exercise with logical error(s)': [27, 25, 42, 35, 26], 'Ended
exercise in correct state': [34, 36, 25, 23, 24], "Didn't attempt exercise": [1,
1, 0, 0, 0], 'Total attempts': [71, 70, 68, 60, 51]}

Number of students who ended every exercise in a correct state: 8
Number of exercise attempts where every run was unsuccessful: [10, 33, 22, 22,
8, 95]

Median edit distances between runs: [[1.0, 1.0, 2.0, 5.0, 33.0], [1.0, 2.0, 3.0,
7.0, 130.0], [1.0, 1.0, 2.0, 4.0, 297.0], [1.0, 1.0, 2.0, 4.0, 325.0], [1.0,
2.0, 3.0, 8.0, 313.0], [1.0, 1.0, 2.0, 5.0, 325.0]] (skewness = [2.07, 4.76,
5.84, 9.98, 4.2, 7.99])
Edit distance between first and last snapshot for each exercise: [[1.0, 7.0,
22.0, 26.0, 33.0], [1.0, 15.5, 29.0, 34.0, 133.0], [1.0, 6.0, 8.0, 27.25,
289.0], [1.0, 7.0, 11.0, 18.75, 85.0], [2.0, 5.5, 10.0, 27.0, 274.0], [1.0, 7.0,
13.0, 29.0, 289.0]] (skewness = [-0.35, 2.18, 4.84, 2.01, 2.33, 4.55])
Median edit distance between first and last snapshot of each attempted exercise:
[[1.0, 7.0, 22.0, 26.0, 33.0], [1.0, 15.5, 29.0, 34.0, 133.0], [1.0, 6.0, 8.0,
```

```
27.25, 289.0], [1.0, 7.0, 11.0, 18.75, 85.0], [2.0, 5.5, 10.0, 27.0, 274.0],
[1.0, 7.0, 13.0, 29.0, 289.0]] (skewness = [-0.35, 2.18, 4.84, 2.01, 2.33,
4.55])
```

1.5 Statistics for the categorisation

1.5.1 First move

[7]:
```
first_time_runs =␣
 ↪get_first_run_time(filtered_ids=filter_program_logs_that_ran_before_changes(),␣
 ↪**dataframes)
print(f"Median first run time for run-first exercises:␣
 ↪{quartiles_for_exercise(first_time_runs)[5][2]:.2f} (skewness =␣
 ↪{skewness_for_exercises(first_time_runs)[5]:.2f})")

time_between_runs =␣
 ↪get_time_between_runs(filtered_ids=filter_program_logs_that_ran_before_changes(),␣
 ↪**dataframes)
print(f"Median time between runs for students who ran first:␣
 ↪{quartiles_for_exercise(time_between_runs)[5][2]:.2f} (skewness =␣
 ↪{skewness_for_exercises(time_between_runs)[5]:.2f})")

time_between_first_and_second_run =␣
 ↪get_time_between_two_runs(filtered_ids=filter_program_logs_that_ran_before_changes(),␣
 ↪run1=1, run2=2, **dataframes)
print(f"Median time between first and second run for students who ran first:␣
 ↪{quartiles_for_exercise(time_between_first_and_second_run)[5][2]:.2f}␣
 ↪(skewness = {skewness_for_exercises(time_between_first_and_second_run)[5]:.
 ↪2f})")

print(f"Number of distinct students who ran before changing in at least one of␣
 ↪the debugging exercises:␣
 ↪{length_of_participants_list(filtered_ids=filter_students_who_ran_before_changes(),␣
 ↪**dataframes)}\n")

unsuccessful_after_initial_run =␣
 ↪get_number_unsuccessful_snapshots_after_given_run(filtered_ids=filter_program_logs_that_mad
 ↪run_number=1, **dataframes)
print(f"Number of change-first programs that still weren't running after␣
 ↪initial run: {unsuccessful_after_initial_run[5]}")

first_time_change_first =␣
 ↪get_first_run_time(filtered_ids=filter_program_logs_that_made_changes_before_running(),␣
 ↪**dataframes)
```

```
print(f"Median first run time for change-first exercises:␣
  ↪{quartiles_for_exercise(first_time_change_first)[5][2]:.2f} (skewness =␣
  ↪{skewness_for_exercises(first_time_change_first)[5]:.2f})")
```

```
Median first run time for run-first exercises: 16.81 (skewness = 4.26)
Median time between runs for students who ran first: 0.92 (skewness = 5.90)
Median time between first and second run for students who ran first: 22.37
(skewness = 4.38)
Number of distinct students who ran before changing in at least one of the
debugging exercises: 25

Number of change-first programs that still weren't running after initial run: 92
Median first run time for change-first exercises: 62.94 (skewness = 1.47)
```

1.5.2 Positive debugging behaviours

[8]:
```
print(f"Number of students who made unsuccessful change to line 4 of exercise 4:
  ↪ {number_students_unsuccessful_first_change_exercise_4(**dataframes)}\n")
```

```
Number of students who made unsuccessful change to line 4 of exercise 4: 31
```

1.5.3 Introduced additional errors

[9]:
```
print(f"Number of exercise attempts who added and did not resolve a syntax␣
  ↪error:␣
  ↪{length_of_program_logs_list(filtered_ids=filter_students_who_added_and_did_not_resolve_syn
  ↪**dataframes)}")

time_in_syntax_error_state =␣
  ↪get_total_time_in_specific_error_state(filtered_ids={}, error_type='syntax',␣
  ↪**dataframes)
print(f"Median time that students were in syntax error state for:␣
  ↪{quartiles_for_exercise(time_in_syntax_error_state)[5][2]:.2f} (skewness =␣
  ↪{skewness_for_exercises(time_in_syntax_error_state)[5]:.2f})\n")

print(f"Number of students who introduced at least one logical error in at␣
  ↪least one of the exercises:␣
  ↪{length_of_participants_list(filtered_ids=filter_students_who_introduced_logical_errors(),␣
  ↪**dataframes)}")
```

```
Number of exercise attempts who added and did not resolve a syntax error: 75
Median time that students were in syntax error state for: 87.73 (skewness =
1.59)

Number of students who introduced at least one logical error in at least one of
the exercises: 59
```

1.5.4 Resolved errors

[10]:
```
print(f"Number of students who correctly resolved at least one error:␣
 ↪{length_of_participants_list(filtered_ids=filter_students_who_correctly_resolved_any_error(
 ↪**dataframes)}")
print(f"Number of students who resolved at least one of the logical errors in␣
 ↪at least one of the debugging exercises:␣
 ↪{length_of_participants_list(filtered_ids=filter_students_who_resolved_logical_errors(),␣
 ↪**dataframes)}")
print(f"Number of students who resolved at least one error by introducing a␣
 ↪logical error:␣
 ↪{length_of_participants_list(filtered_ids=filter_students_who_introduced_logical_error_thro
 ↪**dataframes)}")
print(f"Number of students who resolved at least one error in a hard coded␣
 ↪fashion:␣
 ↪{length_of_participants_list(filtered_ids=filter_students_who_hard_coded_resolved_any_error
 ↪**dataframes)}\n")

print(f"Number of students who made same first change as Bukayo on exercise 1:␣
 ↪{number_students_first_change_same_as_student_19_exercise_1(**dataframes)}\n")
print(f"Number of students who ended exercises with one or more logical errors,␣
 ↪given that they had introduced at least one logical error:
 ↪\n{get_end_state_of_program(filtered_ids=filter_students_who_made_shortcut_changes(),␣
 ↪**dataframes)}\n")
```

```
Number of students who correctly resolved at least one error: 62
Number of students who resolved at least one of the logical errors in at least
one of the debugging exercises: 56
Number of students who resolved at least one error by introducing a logical
error: 42
Number of students who resolved at least one error in a hard coded fashion: 30

Number of students who made same first change as Bukayo on exercise 1: 15

Number of students who ended exercises with one or more logical errors, given
that they had introduced at least one logical error:
{'Ended exercise with syntax error(s)': [3, 20, 16, 9, 7], 'Ended exercise with
runtime error(s)': [11, 5, 0, 13, 0], 'Ended exercise with type error(s)': [2,
4, 2, 18, 0], 'Ended exercise with logical error(s)': [26, 20, 29, 24, 15],
'Ended exercise in correct state': [11, 16, 10, 9, 12], "Didn't attempt
exercise": [1, 1, 0, 0, 0], 'Total attempts': [44, 43, 40, 35, 28]}
```

1.5.5 Miscellaneous behaviours

[13]:
```
print(f"Total number of repeated runs:␣
  ↪{overall_total(get_number_unchanged_runs(filtered_ids={}, **dataframes))}")
print(f"- Total number of repeated unsuccessful runs:␣
  ↪{overall_total(get_number_unchanged_unsuccessful_runs(filtered_ids={},␣
  ↪**dataframes))}")
print(f"- Total number of repeated successful runs:␣
  ↪{overall_total(get_number_unchanged_successful_runs(filtered_ids={},␣
  ↪**dataframes))}\n")

print(f"Number of repeated successful runs by exercise number:␣
  ↪{total_per_exercises(get_number_unchanged_successful_runs(filtered_ids={},␣
  ↪**dataframes))}")
print(f"Median number of unchanged runs per exercise:␣
  ↪{quartiles_for_exercise(get_number_unchanged_runs(filtered_ids={},␣
  ↪**dataframes))[5]} (skewness =␣
  ↪{skewness_for_exercises(get_number_unchanged_runs(filtered_ids={},␣
  ↪**dataframes))[5]:.2f})\n")

print(f"Median time between unchanged runs:␣
  ↪{quartiles_for_exercise(get_time_between_all_unchanged_runs(filtered_ids={},␣
  ↪**dataframes))[5]}")
time_between_unsuccessful_unchanged_runs =␣
  ↪get_time_between_unsuccessful_unchanged_runs(filtered_ids={}, **dataframes)
print(f"Median time between unchanged unsuccessful runs:␣
  ↪{quartiles_for_exercise(time_between_unsuccessful_unchanged_runs)[5]}␣
  ↪(skewness =␣
  ↪{skewness_for_exercises(time_between_unsuccessful_unchanged_runs)[5]:.
  ↪2f})\n")
all_times_between_unsuccessful_unchanged_runs =␣
  ↪time_between_unsuccessful_unchanged_runs[5]
unsuccessful_unchanged_runs_over_10_seconds = [time for time in␣
  ↪all_times_between_unsuccessful_unchanged_runs if time > 10]
print(f"Number of repeated unsuccessful runs more than 10 seconds apart:␣
  ↪{len(unsuccessful_unchanged_runs_over_10_seconds)}␣
  ↪({len(unsuccessful_unchanged_runs_over_10_seconds) /␣
  ↪len(all_times_between_unsuccessful_unchanged_runs) * 100:.2f}%)\n")

time_between_successful_unchanged_runs =␣
  ↪get_time_between_successful_unchanged_runs(filtered_ids={}, **dataframes)
print(f"Median time between unchanged successful runs:␣
  ↪{quartiles_for_exercise(time_between_successful_unchanged_runs)[5]}␣
  ↪(skewness =␣
  ↪{skewness_for_exercises(time_between_successful_unchanged_runs)[5]:.2f})")
all_times_between_successful_unchanged_runs =␣
  ↪time_between_successful_unchanged_runs[5]
```

```
successful_unchanged_runs_over_10_seconds = [time for time in␣
 ↪all_times_between_successful_unchanged_runs if time > 5]
print(f"Number of repeated successful runs more than 10 seconds apart:␣
 ↪{len(successful_unchanged_runs_over_10_seconds)}␣
 ↪({len(successful_unchanged_runs_over_10_seconds) /␣
 ↪len(all_times_between_successful_unchanged_runs) * 100:.2f}%)\n")

print(f"Number of students who repeatedly successfully ran their program in␣
 ↪exercise 5: {len([x for x in␣
 ↪get_number_unchanged_successful_runs(filtered_ids={}, **dataframes)[4] if x !
 ↪= 0])}")
print(f"Median time between successful unchanged runs for exercise 5: ␣
 ↪{quartiles_for_exercise(get_time_between_successful_unchanged_runs(filtered_ids={},␣
 ↪**dataframes))[4]} (skewness =␣
 ↪{skewness_for_exercises(get_time_between_successful_unchanged_runs(filtered_ids={},␣
 ↪**dataframes))[4]})\n")
```

```
Total number of repeated runs: 4809
- Total number of repeated unsuccessful runs: 2283
- Total number of repeated successful runs: 2526

Number of repeated successful runs by exercise number: [82, 22, 121, 129, 2172,
2526]
Median number of unchanged runs per exercise: [0.0, 2.0, 5.0, 18.0, 213.0]
(skewness = 4.01)

Median time between unchanged runs: [0.01, 0.19, 0.38, 0.98, 345.19]
Median time between unchanged unsuccessful runs: [0.02, 0.19, 0.37, 1.01,
345.19] (skewness = 7.86)

Number of repeated unsuccessful runs more than 10 seconds apart: 276 (12.09%)

Median time between unchanged successful runs: [0.01, 0.19, 0.38, 0.95, 222.18]
(skewness = 9.21)
Number of repeated successful runs more than 10 seconds apart: 265 (10.49%)

Number of students who repeatedly successfully ran their program in exercise 5:
40
Median time between successful unchanged runs for exercise 5:  [0.01, 0.19,
0.32, 0.7, 222.18] (skewness = 16.4)
```